Data Transparency

I assume here that I do not need to spend a lot of time introducing concepts like Data Transparency and open-access data sets. Long story short, increasingly the basic unit of dissemination for academic scholarship and scientific research is not the individual text publication, but a package that can contain data, text, computer code, and multimedia presentations.

"Replication Crisis"

The "Replication Crisis" that became recognized in the 2010s has surely contributed to this development because scientists wanted to evaluate their peers' research more thoroughly. Instead of publications just providing summaries, scientists want to examine raw data files, data acquisition protocols, data-transformation pipelines, and analytic methods.

Data Transparency Becomes a Priority

Some publishers have started to *require* authors to provide raw data alongside their articles, or else to explicitly state why they are unable to do so (assuming their research involves empirical data sets to begin with). Similar policies have been adopted by funding agencies. For example, the Bill and Melinda Gates foundation requires that all grantees "include a Data Availability Statement (DAS) describing where the underlying data can be found ... Underlying data includes all primary data, metadata, and any additional information needed to understand, assess, and replicate the study findings."

Gates Foundation 2025 Open Access Policy: Data Availability Guide

MIBBI and Other Minimum Information Standards

Data Transparency goes beyond just sharing data once it is acquired. For scientific fields where observations are obtained from physical or laboratory equipment, properly assessing and reproducing research requires relatively detailed understanding of the experimental setup prior to data being captured. Minimum Information standards allow these details to be notated in a structured manner conformant to domain-specific templates. By extension, the overall procedures through which samples and information are transformed — both in the physical and digital/computational realms — should be documented with an eye toward replication.

"Findable, Accessible, Interoperable, Reusable"

According to the popular "FAIR" standard, data should be Findable, Accessible, Interoperable, and Reusable. Arguably the most important consideration here is accessibility. In the typical case, a researcher will learn about some scientific work or experiment from peer-reviewed literature. Once they are reading a specific publication — and should they wish to examine the research in greater detail — the question becomes how readily they can acquire the associated raw data and examine it usefully.

Interoperability means that multiple data sets can be studied and analyzed side-by-side, and Reusability means that data and code can be reused in new experiments — maybe reproducing and/or extending the original data set. But accessibility is a precondition for everything that follows.

Data Sets

Accessibility has multiple dimensions. Starting from a published manuscript, there are usually several steps required to download, deserialize, and study data sets. With FAIR data, these steps can hopefully be accomplished without relying on a lot of external software or manual processing. Ideally, computer code needed to unpack and visualize data contents are included as part of a data set.

FAIR data sets should not rely on users having access to external software, especially software that is not free and open-source. Researchers should be able to assess scientific work without having the same applications or tools that the original scientists employed. For most data sets, raw data needs special deserialization and visualization tools. For FAIR/Accessibility, XML files can't just be viewed as DOM trees in web browsers, and CSV can't just be opened as spreadsheets.

Executable Research Objects

As a standard related to FAIR data, the Research Object specification includes formats such as Research Object Bundles and RO-Crate that govern how data packages are organized and what kinds of metadata they provide. These formats typically include one or more structured files that provide a machine-readable summary of data contents, plus raw data files organized for straightforward access.

More generally, "Research Objects" can refer to any data set published according to FAIR principles, which means that raw data is supplemented with metadata and code to facilitate accessibility, interoperability, and reuse. An "Executable" Research Object prioritizes included code in particular. In particular, an overall data set can be developed as a self-contained computer application with a dedicated entry-point that provides access to different parts and facets of the Research Object. That is, RO code is aggregated into a unified executable package.

Publications Within Research Objects

Individual text publications are still important because they are usually the place where readers learn about research work — e.g., either via a web search or reference in a different publication. But once someone accesses a manuscript that is linked to a Research Object, they may then download that data set and conduct further reading and examination through the Research Object directly. If the original paper is included within that Research Object — in a format such as PDF — readers can view the manuscript via the included file rather than the web document they may have found initially. This creates new possibilities for annotations, cross-references, and interactive content, insofar as words, sentences, and other landmarks in article texts can be linked to data and code elsewhere in the Research Object.

What's "Cross-Reference Logic"?

Note that for a PDF rendering library, each PDF page is just a picture, which can be manipulated and overlaid with generic graphics functionality. The window coordinates of textual elements can be obtained from PDF annotations or by pairing PDF files with metadata compiled using techniques such as `\pdfsavepos`. This means that context menus activated over a PDF page can have different options depending on the contents of words, sentences, or annotations present near or around the mouse-event position. Also, the PDF display can be embedded in GUI windows with additional buttons and controls. Whether via context menus, toolbars, or other controls, Research Object Applications can link PDF windows programmatically with other windows showing videos, digital maps, computer simulations, 3D graphics, and so forth. PDF libraries such as Poppler or XPDF can be included directly in Research Objects, so PDF viewers can be customized for individual data set and shared directly as source-code files.

Text Mining

Alongside the emergence of open-access data sets and Research Objects, there has also been an increasing emphasis on text-mining and AI techniques to expedite academic research. The basic idea is improving the accuracy of search results, by leveraging computationally tractable semantic cues that are more granular than simply searching for specific text strings, or even thematically related text. For example, there are thousands of peer-reviewed articles about Keynesian Economics, and merely identifying some publication as relevant to this overall topic does not clarify whether that document is important to some *particular* researcher's work about this topic. Instead, we'd like to discover more specific classifications, such as resources that *endorse*, *critique*, or neutrally *evaluate* Keynesian Economics; or analyze it from a macroeconomic, or a sociopolitical perspective; or assess it differently from a moral versus quantitative perspective, and so on.

Case Study: "CORD-19"

The CORD-19 corpus, developed by the Allen Institute for Artificial Intelligence (AI2), is a good example of the goals and techniques for text-mining in the context of scientific research. This corpus included over 400,000 freely-available full-text articles available both in human-readable formats and in machine-readable encodings — specifically, S2ORC JSON (S2ORC comes from Semantic Scholar Open Research Corpus). Because Covid research spanned many disparate subject areas — viral morphology, infectivity, genetic profiles and strains, vaccine development, diagnostics, clinical treatments, epidemiology, etc. — automated text-mining tools could, at least in principle, help scientists discover research in other disciplines to help fit the pandemic's pieces together. AI2 did not specifically create those tools, but they curated CORD-19 to be directly accessible from computer software in the hopes that other parties could build text-mining capabilities upon that basis.

Limitations of CORD-19

According to AI2, roughly 30% of CORD-19's full-text documents were shared in structured formats such as JATS-XML, but the majority of publications were only available as PDF files. AI2 therefore had to extract text from those PDFs in order to perform conversion to S2ORC JSON. This is an intrinsically error-prone process due to anomalies within internal PDF text representation, such as hyphenation, ligatures, spurious space characters in some font styles, headers/footers and marginal content that needs to be isolated from the main text, etc. Furthermore, technical content such as chemical formulae or linguistics annotations can be difficult to read properly without working off of structured markup, such as XML. Transcription errors limited the extent to which inter-document connections and conceptual links can be properly recognized. Also, S2ORC only models documents at the paragraph level — not individual sentences — which can limit the application of NLP-related text mining techniques.

AI2 recognized these limitations and even printed a "call to action" encouraging publishers to develop more rigorous text encoding and metadata standards.

But Let's Not Get Too Locked In on Text Mining

I think it is hard to judge the prospects of text mining as a research tool in these ways, sensitive to rhetorical structures and NLP nuance. Nonetheless, the emphasis on machine-readable text encoding — which has been spurred on by text-mining goals — is also important in other contexts.

Even without AI or NLP, rigorous text representation can improve upon native PDF search capabilities; help to implement text-to-data cross-references; and facilitate systematic, actionable semantic annotation schemes within published manuscripts. For instance, intra-document queries can be narrowed in scope to the main text; or, conversely, to block quotes, footnotes, section titles, etc. Moreover, special content such as chemical formulae, technical definitions, linguistics example sentences, proper names, GEO-taggable place names/locations, quotations, citations, and etc. can be parsed into structured data objects.

Sentence Objects

There are numerous XML-based and other markup formats for text documents, including JATS, BioC, TEI, S2ORC, and TAGML. Any markup format, however, will encode natural-language text according to generic structures involving trees or graphs, rather than the inherent structure of written text itself. For instance, XML trees are defined in terms of parent and child nodes, siblings, elements' text content, and attribute values. Although there are analogous structures in natural language — for instances, sentences in sequence are similar to sibling XML nodes many textual phenomena can be modeled only partially via XML structure alone.

For these reasons, any markup file should be seen as the *serialization* of textual content, not as the definitive structure of that content. Conceptually, text elements are best understood as typed values in an Object-Oriented context. Moreover, I believe sentence objects — that scale of representation — should be the foundation of such object systems. Sentence objects are also contained inside larger elements (paragraphs, sections, subsections) and can encompass smaller parts (keywords, annotated fragments), but overall text-object systems should pivot around the sentence level.

Sentence Objects and Query Results

This slide shows examples of how text can be modeled in terms of sentence objects. In this specific case, the text for the full-length article I am summarizing with these slides — which is programmed to generate both LaTeX and JATS files — also creates serialized versions of sentence objects. The resulting data files can then be parsed — in this case, into C++ objects, objects that can then be used for editing and text analysis. I have found this a useful system when preparing books and articles for publication. On this slide, I show how the objects can be employed for a demo full-text query system. The query is presented within demo code published for this article. This is not a full-fledged query language, but a provisional outline of the technology stack and implementation layers supporting such a language.

Text Elements as Structured Objects

I use the term "Sentence Object Model" to describe text encoding built around sentence objects, but where objects can encompass a spectrum of distinct types both at higher and lower scales. Among sentence objects themselves, there is usually a straightforward structure — where sentences follow one another in sequence, belonging to encompassing paragraphs — but sometimes the structure is more complex. For instance, footnotes may contain multiple sentences or even paragraphs, but are in some sense children of the sentences where their mark appears — or, if it comes after a sentence, the sequence of sentences effectively diverges into main text and footnote content. Similarly, a block quote or enumerated list often functions like a child of the sentence that introduces it, and sometimes the preceding sentence is grammatically incomplete and has the effect of merging with the content that follows it. Also, special content such as quotations, citations, bibliographic entries, and entries/locators for print indexes have their own structural requirements. As these examples suggest, modeling the full organization of textual content requires an ability to represent sometimes complex objects and their interrelationships.

Para Ids and Sentence Boundaries

This slide and the next show a sample document that illustrates one way to use paragraph subdivisions and explicit sentence boundaries. Publishers have started to use paragraph ids instead of page numbers to record index locators during manuscript preparation. The obvious benefit of para-id's is that they remain constant even while pages get renumbered due to changes elsewhere in the document. They are also more granular than page numbers alone, directing us to specific parts of a page — especially in conjunction with 2-column text.

Ordinarily, para-ids are replaced by page numbers in the final publication and are not visible to human readers. However, it is possible that future publishers will choose to keep para-ids as marginal content, to help human readers locate and refer to specific locations on a page. Visible ids introduce hyperlink sources and targets that can lead back and forth between the index and the main text.

Continuation Marks: a, b, c; i, ii, iii; left/right column

With respect to para-ids, one nontrivial question is to define the scope of individual paragraphs. Many paragraphs can include identifiable parts, such as a block quote, followed by further discussion, then perhaps another quotation. This sample document uses a sub-indexing scheme to identify sequential parts of a paragraph within the main text (via letters **a**, **b**, **c** etc.) and block quotes inside paragraphs (via roman *i*, *ii*, etc.). Moreover, special marks identify portions of paragraphs that run onto a new page or the top of a page's right-hand column.

This graphic also shows special marks added to identify sentence boundaries — or, more precisely, the boundaries that have been modeled via metadata. Such marks ordinarily would not be visible to typical readers, but they could be used by authors, editors, or programmers to double-check sentence boundaries as computationally recognized.

Paragraph Subdivisions and Index Locators

This slide shows the index of the same document. Ordinarily articles do not have a print index — only books — but that convention too might change as text-encoding becomes more rigorous. Here, the index locators are provided both via page numbers and via para-ids with subdivisions. Clicking on page numbers scrolls to the top of the relevant page, whereas clicking on the ids brings up the corresponding page just above the desired paragraph (or subdivision thereof).

On the left-hand side of this page is a list of example sentences discussed in the paper — a common pattern for linguistics articles. I believe it should be adopted as a common practice in this discipline to have all such samples reproduced at the end of a document, similar to a glossary. Moreover, example sentences can be exported as metadata and merged into linguistic corpora. Similar techniques could be used for other academic fields wherein publications have special kinds of segments or environments that could be coalesced into glossary-like structures.

GUI Controls for (Print) Indexing

Once text is compiled into sentence objects — represented, for example, by C++ classes — then the C++ data may be queried to support document-preparation tasks, including editing, checking references, and compiling an index. Ideally, queries are routed both to internal PDF searches and to text/element objects.

"Indexing" in this context does not refer to construction of a web index — which is technically a "reverse" index, mapping key words to the set of documents that contain them (or contain semantically similar words). Reverse indexes are typically machine-generated and provide only an approximate model of documents' content based on word- or concept-occurrence vectors. A print index, by contrast, is manually compiled and reflects the authors' domain-expert understanding of which terms deserve separate index entries, and which paragraphs are relevant matches for those concepts.

GUI Window for Single Index Entry

Even one single index entry can be relatively complex, with multiple locators — that is, multiple pages or paragraphs notated as a conceptual match — plus potential "see" and "see also" redirection, and/or potential subentries. During document prep, it is entirely possible to implement special-purpose GUI windows specifically dedicated to individual index entries.

For any given project, it may be an open question whether the development effort needed for general-purpose indexing GUIs is worth the added convenience, because some users might actually find it easier to work with text files serializing lists of index entries. Nonetheless, a reusable suite of indexing GUI tools could certainly be part of a comprehensive software package for Executable Research Objects.

Meta-Index Search

Once authors or editors have expended the effort to create print indexes for individual publications, a natural extension of that labor would be to pool indexes of related documents, such as those published within a single journal or book series. Meta-index web services could then be accessed from individual publications via PDF extensions (as well as accessed through ordinary web portals).

Meta-Index searching can be contrasted with — and performed alongside — broader reverse-index searches. On the one hand, meta-indexes are narrower in scope because they presumably apply to smaller document collections. This would be particularly true if meta-indexes were combined with full-text representations that could be searched as rigorously as local files, without the simplification and data-compression of reverse indexes (which use word stemming/lemmatization and generally discard sentence contexts). On the other hand, meta-index searches can be more granular and contextually fine-tuned.

Reverse-Index Searches

In contrast to meta-indexes, the reverse-index lookup employed by search engines is a much older technology. Nonetheless, it might be useful to design GUI windows specifically organized to initiate queries against conventional full-text search engines.

Full-text queries often have multiple input parameters that can adjust how the search operates. Usually these details are invisible to end-users, but on some occasions users might want to exercise fine-grained control over search policies. This might be achieved via HTML forms in the context of web services, but dedicated GUI windows could be designed instead in the context of native-compiled document-prep tools. For example, these searches could help authors or editors find relevant publications for citations and bibliographies.

Viewing Sentence and Annotation Boundaries

Representing text at the sentence level allows queries to be evaluated that focus on individual sentences as containers and/or the basis for reporting query results. For example, queries could search for words or phrases with the stipulation that they appear specifically at the beginning of a sentence. Or, one could locate the first sentence in a document, or a chapter, where the search text appears.

This slide shows functionality that could be enabled when sentence-object data is read by code within a PDF viewer. Here, the user has activated a context menu to read the contents of the current sentence (a related context option would be to copy the current sentence to the clipboard). The resulting dialog window also shows the locations of proper names and annotations within the current sentence.

Data Objects for Executable Research Objects

So thus far I have talked about an Object-Oriented model for sentences and other textual elements, and also about Research Objects and data sets. We can assume that, in many cases, Executable Research Object code will be working with both kinds of data — that is, textual objects obtained from sentence-based encoding and data objects deserialized from raw data files. The Research Object code will need capabilities to navigate, examine, and search against collections of objects whose class types cover both text and research data. Merging these forms of data into a common information space affects several dimensions of Research Object implementation, including searching, coding practices, documentation, and GUI design. We can consider how search and semantic indexing techniques can be extended to data types, fields, and procedures implemented by dataset code.

Cross-References (again)

With respect to searching, in some contexts the same semantic models that apply to natural language carry over to structured data objects, at least in some facets. For example, any data structure that can be arranged into tables has a notion of column labels, and their terminology may well match phrases present in associated publications. Numeric quantities are often expressed in units of measurement, which are also semantic entities — for instance, we might want to search for every field in a data set expressed in latitude/longitude coordinates. Code procedures that accept arguments dimensioned according to specific units of measure are also semantic matches for those units. In the case of GIS, for instance, algorithms that convert to or from other coordinate systems — such as "XYZ" — would be reasonable matches for terms "latitude" and "longitude". In general, semantic indexing can be extended to data objects and computer code.

GUI Indexing for Research Objects

Semantic indexing can also be extended to GUI design. Here are some examples: Any context menu has a list of options, which might be determined by the local object where the menu was activated, and which are all semantic entities. The same is true for controls wherein a user can select one of multiple options, such as QComboBox — each instance is effectively a controlled vocabulary. Many GUI classes have "actions", which are methods that can be explicitly initiated by users — i.e., they represent the handlers for class functionality directly available to users via menus, buttons, and other controls. Another facet relevant for indexing is the different kind of user gestures applicable for a given GUI object, including button-press, dragging, and turning the scroll wheel.

Ideally, Executable Research Objects — like any GUI application — will have a manual or documentation helping user finds specific units of functionality. For instance, any GUI window could be catalogued with a list of controls inside it, their purpose, and how users can interact with them (by clicking, dragging, etc.)

Computational Support for Text/Data Integration

Assuming that we have merged research text and data into a common object system, we can then how to benefit from the resulting information spaces. If we assume that an Executable Research Object is implemented in a language like C++, then provisional access to the object system would come via C++ code. However, scripting and/or query languages offer a different interface to the underlying functionality with some benefits of their own: allowing C++ classes to be reused by multiple data sets (because scripts can provide the finer details for individual projects); for unit testing and documentation (scripts can generate "user manuals" instead of relying on source comments); for implementing text queries by authors or editors without having to write and compile new C++ code. (As one example, "manual-generating" scripts could apply code-introspection techniques together with LaTeX or XML templates.)

Using Scripts to Enhance Research Object Presentations

Supposing an Executable Research Object is primarily coded in C++, we can still use a scripting language to program specific capabilities. For instance, a language could be chosen (or implemented) which is self-documenting and emphasizes explicit interface design more aggressively than C++. Design-by-Contract (DbC) techniques can be employed to explicitly notate procedures' assumptions and preconditions. Units of measurement and coordinate systems can be inherent parts of the type system — e.g., an $\{x, y\}$ pair distinguished from {row, column} or {width, height}. Algorithms or formulae that might be described cryptically in article text can be presented in a more accessible manner within computer code. In general, code libraries can help demonstrate the theories and protocols affecting research work — especially if programming languages are designed to prioritize this pedagogical dimension.

The Trouble with Deserialization

Focusing on deserialization in particular, there are many domain-specific data formats such as FASTQ and VCF for genomics; HDF5 and ROOT for physics; FITS and netCDF (astronomy and earth sciences); PDB, MOL, SBML, and CELLML (biochemistry); IFC and BIM (architecture/engineering); SEGY and BAG (geophysics and oceanography); CoNLLU (computational linguistics); DICOM and OMETIFF (bioimaging); and GIS-indexed attribute layers and shapefiles for geospatial data sets. Whether or not these formats are based on generic standards like XML or JSON, or alternatively require their own special-purpose parsers, it usually requires special code to convert raw data files to usable data objects. It is obviously impractical for everyone to write such code from scratch simply to access raw data files — which is why organizations often provide deserialization library for common use. But such libraries can have limitations, including lags in updating code for new format specifications and difficulties in porting code to different computing environments. In this case, I believe a scripting language built with a lightweight and self-contained technology stack could alleviate some of these problems.

Language Stacks for Research Objects

So these are some use cases for a scripting language specific to Executable Research Objects — and I will also discuss full-text search and how something like this could also serve as a query language. But just to talk about implementation: the idea here is specifically that a language be built *inside* a Research Object, with all the code layers published internally as source files — minimizing external dependencies as much as possible. Certainly we should not rely on heavy, complex dependencies such as LLVM or libClang. The tech stack should be built from the bottom up with an emphasis on functionality specific to Research Objects: deserialization; • Design by Contract as a pedagogical as well as design tool; • documenting scientific theories and data models; • generating manuals and interactive tutorials; • text/data integration. And the language should be customizable, with the option of adding new grammar rules, VM instructions, and built-in runtime functions for each data set.

Parsing for an Embedded Compiler

Conceptually, a compiler pipeline begins by reading input files into a "parse graph". In a schematic sense, each grammar rule presents an opportunity to add a node to the graph. Rules can be checked sequentially at the current "cursor" position — activated or deactivated via contexts — with the first matching rule preempting others (so that rule-order is a structural feature of the grammar). To avoid reliance on external parser-generator tools, each rule can actually be a normal regular-expression string coupled with a callback handler (e.g., anonymous/"lambda" functions). When the graph is complete, it may be subject to transformations — for instance, macro expansion — and then, in its final form, is traversed strategically to generate VM bytecode.

This is a useful outline, but the workflow in practice may be a little different. I have found it expedient to implement a "preliminary" VM whose role is to assemble the parse-graph and/or generate bytecode — in other words, rule callbacks do not create nodes on their own, but rather emit "provisional" VM code. And, in the system discussed here, "bytecode" is actually just methods of specific C++ classes, but designed to emulate things like opcodes, registers, and stack frames.

Rules and Contexts in Parsing Code

This slide shows a little segment of the grammar from demo code I've published alongside this presentation. I want to call attention specifically to the notion of context, which actually occurs here on two levels. Although the rule expressions are based on regular expressions, they are actually preprocessed — allowing a slightly expanded regex format — and then initialized into rule objects that test current contexts. "Outer" contexts serve more general purposes and can be activated together — i.e., one context can trigger others when it comes into effect. For instance, an outer context might be ordinary code versus extended, structured comments for documentation. "Inner" contexts are boolean flags and usually active over a shorter span. For instance, an inner context might be rules active only inside quote literals. Here, the inner context involves statements parsed specifically as full-text queries.

The Idea of "Channels"

Another structure I find it useful to introduce into script-compilers is what might be called "channels". Conceptually, functions in computer code — not too different from mathematics — have zero or more inputs and return zero or more outputs; i.e., there is one tuple for input parameters and one for output results (of course, for many languages we can return at most one value). However, even mainstream languages complicate this picture via "**this**" objects, for instance, that are notated apart from other inputs and may even be passed invisible; and via exceptions, which are detached from typical return-value semantics. These constructions might be seen as distinct input and output channels, and we can generalize this observation: languages can employ special channels to represent arguments that have some semantic or syntactic separation from ordinary parameters and therefore require special compiler or runtime treatment. Implementing a channel system allows these special code-points to be identified and isolated, for static analysis or runtime monitoring. Channels permit flexible, "user-defined" calling conventions.

Building Channels Programmatically or From VM Code

In the demo code, channels are built up incrementally by specific C++ classes. A channel "package" then serves as the container object through which pre-compiled C++ code may be called via function pointers (as well as procedures defined in script). In order to test the channel aggregation and/or foreign-function interface, it is possible to simply call the relevant C++ methods in sequence, from other C++ code — an example of this approach is here on the right. More commonly, of course, the steps to build up channel packages are exposed as VM instructions. An example of VM code designed for this kind of system — again, taken from the paper's demo code — is on the left.

Hidden or "Back Door" Channels

An invisible "this" object is one example, but there are many cases where caller and callee procedures can communicate via some "hidden" channel — that is, parameters are passed in to functions without being explicitly notated in the list of arguments. This is one way to implement Design by Contract. Suppose a procedure takes two lists, which must be the same size. An explicit test like *if*$(x.size() == y.size())\{...\}$ can ensure that this condition is met within the "true" branch of the *if* (assuming, of course, the lists are not modified therein). So, the compiler could insert a "proof" object into the lexical scope of that branch, which could then be silently passed to the DbC procedure. Conversely, if the programmer can guarantee that the same-size condition is met — e.g., if the second is built from the first somehow — then they could explicitly notate such a proof in the current scope and then allow it to be similarly "back-doored".

Another example is passing a pre-allocated (but not initialized) memory buffer for "placement new" when the caller cannot provide a default-initialized mutable reference and does not want a heap-allocated return value.

Interface Design Principles for the Research Object Context

Assuming that a portion of Research Object code is a library developed in a scripting language specially engineered for data sets, we can then consider how type and procedural interfaces could be designed for such a language. The design patterns need not simply reiterate "host" languages like C++. For example, a common question in C++ is whether to accept references or return pointers, for large/complex return values. A streamlined interface protocol could allow procedures to adopt both conventions — plus placement-new initialization just mentioned — depending on channel-package construction. Another interesting feature is "projecting" declared variables to take on the lifetime of either an inner, nested scope or outer, containing scope — which is particularly relevant when variables are declared in the course of aggregate expressions. Meanwhile, "Overload by Contract" is a way to implement DbC by requiring multiple versions of procedures, depending on the status of contract tests (pass, fail, or unknown). And intrinsic support for pattern-matching over graph-like or type-reflection structures is a technique for embedding query expressions directly into the language — including text-query for sentence objects.

Diamond Open Access

The "Diamond" Open Access model assumes that no fees are charged for either authors or readers. Often, authors retain full rights and control over their work. This model is often endorsed in terms of fairness and inclusion, but it has technological dimensions as well: programming data sets and Executable Research Objects is much easier when the textual and raw-data content shares the same licence and copyright. In particular, ideally Research Objects will include PDF versions of associated publications — and may have a custom PDF viewer with special features just for that paper — as well as text-encoding of the same content in formats like JATS or serialized sentence objects. Assuming everything is above-board, this is untenable unless text documents and data sets share the same license permissions.

In lieu of Article Processing Charges, authors might take on more document-preparation responsibilities for themselves, or collaborate with others who are specialists in digital editing and related skills. This can be appropriate when preparatory work involves not just text manuscripts but also data sets and Research Object code.

Translational Science and Translational Medicine

The notion of "Translational Science" originated with "Translational Medicine", or "bench to bedside": how to translate research breakthroughs into clinical interventions that improve patient outcomes and experience. Beyond just medicine and biology, we now have "translational chemistry", "translational engineering", "translational sustainability", "translational humanities", and so on. This is relevant for publishing because it affects the makeup of research teams: translational emphases imply diverse, interdisciplinary groups including some individuals whose specializations involve concrete implementation of research work, which can be affected by social, cultural, linguistic, economic, and logistical factors. Traslationality can imply people working "on site" for data acquisition, community outreach, language interpreters, nursing or social work, etc. Within such interdisciplinary teams, it is possible for one or more individuals to take particular responsibility for curating data sets and research publications — instead of deferring to publishers' APC-driven internal workflows or hiring external copy/digital editors. Publishers should also adjust to the social realities of a genrea scholarship reflecting practical, community-based experience and observations as well as academic research in labs and libraries.

Science/Humanities Integration

With "Translational Science" as a foundation, let's reiterate the principle that natural sciences and humanities are intrinsically linked in some places, and they reflect a continuum of overlapping disciplines more than a sharp divide. Many aspects of computer science, in particular, embody humanistic as well as scientific themes. Computer programming languages, for example, encapsulate some principles of natural language. The two are indeed very different — my point is not that linguists should help design programming languages because people somehow "talk to" computers — but still, the criteria for how computer languages are clear and expressive to human coders, and how they can be internally implemented, involve forms of structural reasoning characteristic of linguistics and other humanistic disciplines more than empirical sciences and mathematics. Humanities scholars should be encouraged to embrace digital/computational disciplines related to programming and data curation, and in this vein can become specialists in document preparation with rigorous computational skills as well as being adept at writing, editing, and other prerequisites for traditional publishing. Interdisciplinary collaborations and academic/community partnerships can provide the joint effort to buttress DOA models.

Thanks

Thanks for listening!

If you'd like to look at these slides again, I have a URL for them on this slide here. Along with that document is a parallel set of slides with versions of the comments I've made during the presentation, that I'm calling "textovers". This slide also shows a link to my paper for this presentation. That document is available in several formats, and there are additional links on its first page.

If you are interested in looking at the demo code, that link is also available from the paper, as a github repository.